

HOMework 3 TEMPLATE

Use this template to record your answers for Homework 3. Add your answers using $\text{\LaTeX}$ and then save your document as a PDF to upload to Gradescope. You are required to use this template to submit your answers. **You should not alter this template in any way** other than to insert your solutions. You must submit all 17 pages of this template to Gradescope. Do not remove the instructions page(s). Altering this template or including your solutions outside of the provided boxes can result in your assignment being graded incorrectly. You may lose points if you do not follow these instructions.

Instructions to upload code have been provided in the handout.

Instructions for Specific Problem Types

On this homework, you must fill in the box for each problem; please make sure your final answer is fully included in the given space. **Do not change the size of the box provided.** For plot boxes, insert the figures generated by `runner.py` and scale them so that they fit entirely inside the box. For short answer questions you should **not** include your work in your solution. Only provide an explanation if specifically asked. Otherwise, your assignment may not be graded correctly, and points may be deducted from your assignment.

Fill in the blank: What is the course number?

10-703

Problem 0: Collaborators

Enter your team's names and Andrew IDs in the boxes below. If you do not do this, you may lose points on your assignment.

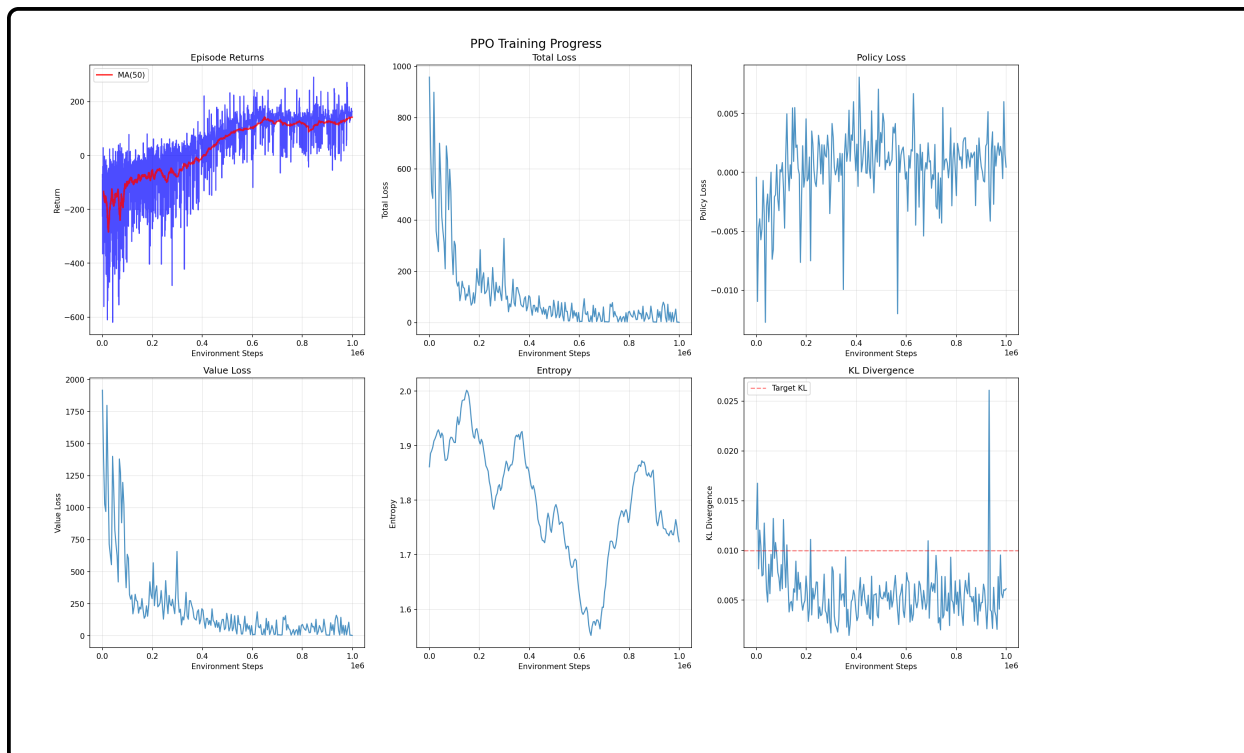
Name 1:	Arnav Goyal	Andrew ID 1:	arnavgoy
Name 2:	Kushal Kejriwal	Andrew ID 2:	kkejriwa
Name 3:	Vineeth Bhat	Andrew ID 3:	vineethb

Problem 1: Proximal Policy Optimization (50 pts)

Problems 1.1–1.3 are graded from your code submission (`ppo_agent.py`) and have no written component.

1.4.1 Clipped Objective

Plots



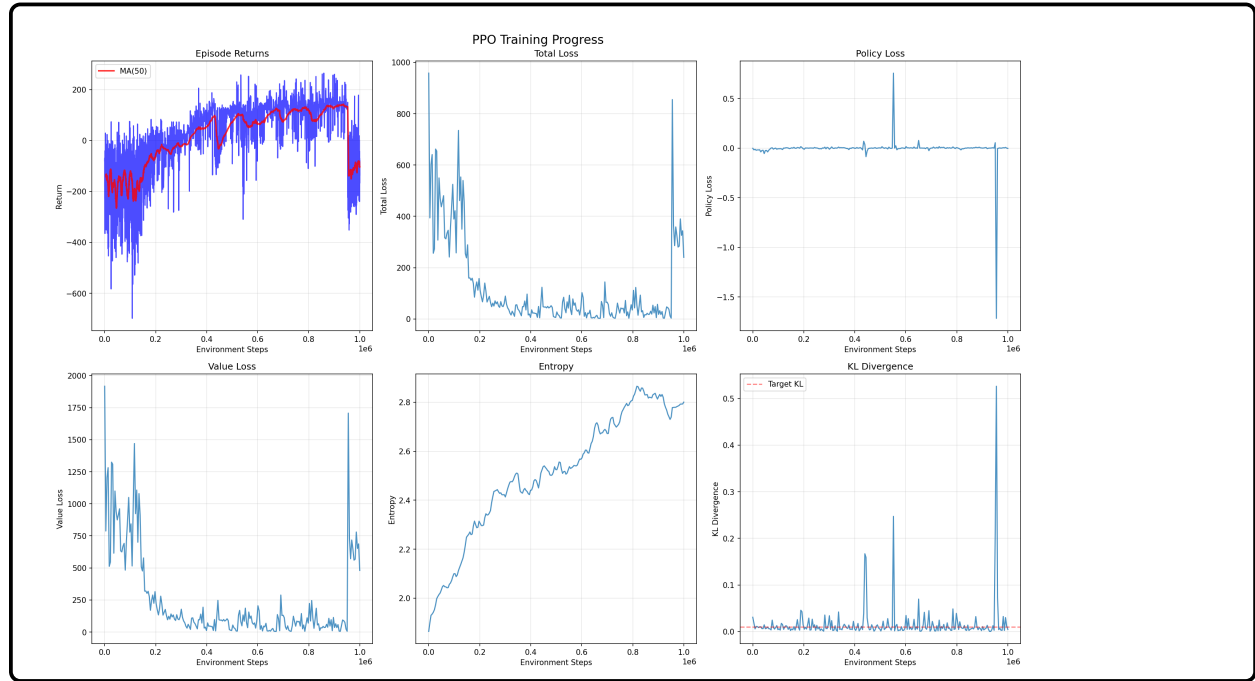
Final performance

Final performance: 214.9 ± 65.4 .

Training: Ending MA(50) = 142.3. Training returns (max/mean/min): all episodes 291.1/ - 56.5/ - 619.4; last 50 271.9/142.3/ - 20.4 (SD 51.5). **Note:** Max > 200.

1.4.2 KL-Penalized Objective

Plots



Final performance

Final performance: 240.1 ± 70.9 .

Training: Ending $MA(50) = -104.5$. Training returns (max/mean/min): all episodes 264.5/ - 74.5/ - 698.4; last 50 18.6/ - 104.5/ - 239.7 (SD 54.2).

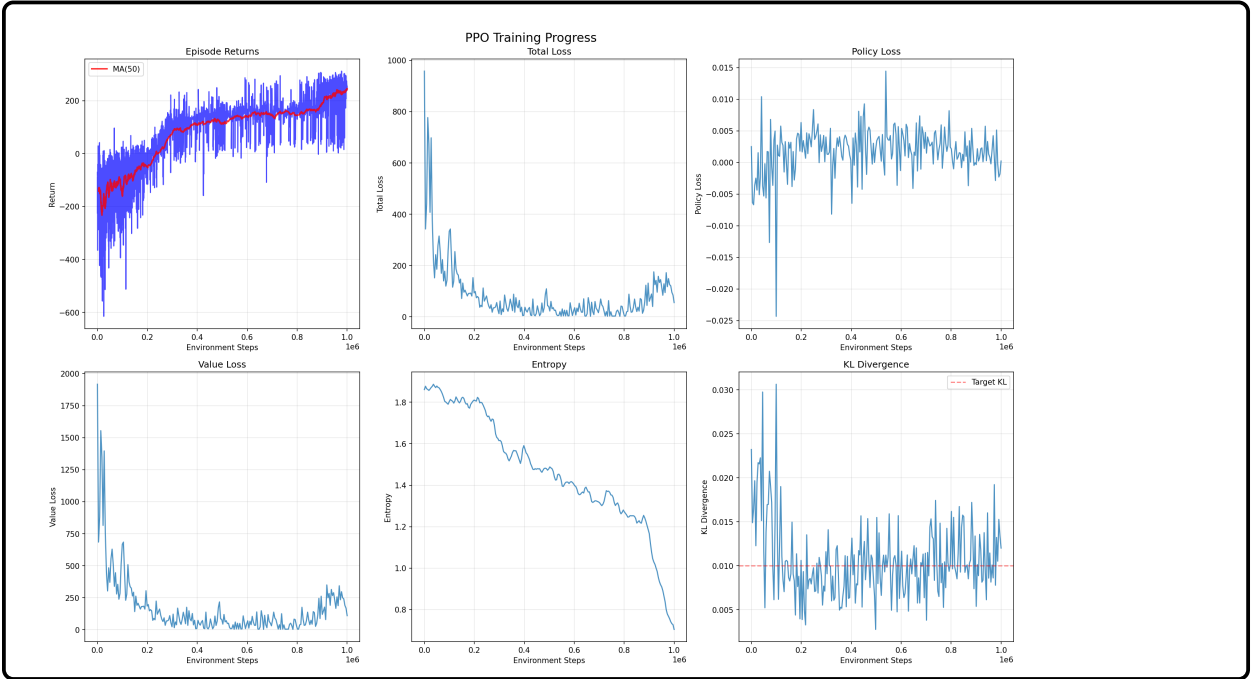
Reflection: clipping vs. KL penalty

Clipping in our trial was more stable in terms of avoiding large KL divergences and large loss spikes. The KL run had a late KL spike above 0.5, as well as sharp increases in policy and value losses while clipping remained relatively stabler. Clipping was also more stable in the sense that it reached a higher $MA(50)$ of 142.3 while KL later collapsed to -104.5 . *However*, KL's best checkpoint evaluated better at 240.1 compared to clipping's 214.9.

Clipping's adv. are simplicity and fewer large KL and loss spikes and better retention of training performance, while its disadv. is that it only removes incentives for excessive probability changes and does not directly constrain overall policy divergence. The KL penalty explicitly discourages divergence and produced a better best checkpoint in our run, but requires adapting β . If the penalty is too weak, disruptive updates can occur and if its too strong, learning can slow. *Neither approach guarantees small updates.*

1.5.1 High Clipping Threshold ($\epsilon = 0.3$)

Plots



Final returns

Final Performance: 254.2 ± 57.0 .

Training: Ending MA(50) = 245.3. Training returns (max/mean/min): all episodes 311.5/2.8/ - 614.8; last 50 302.6/245.3/15.3 (SD 53.9).

Discussion

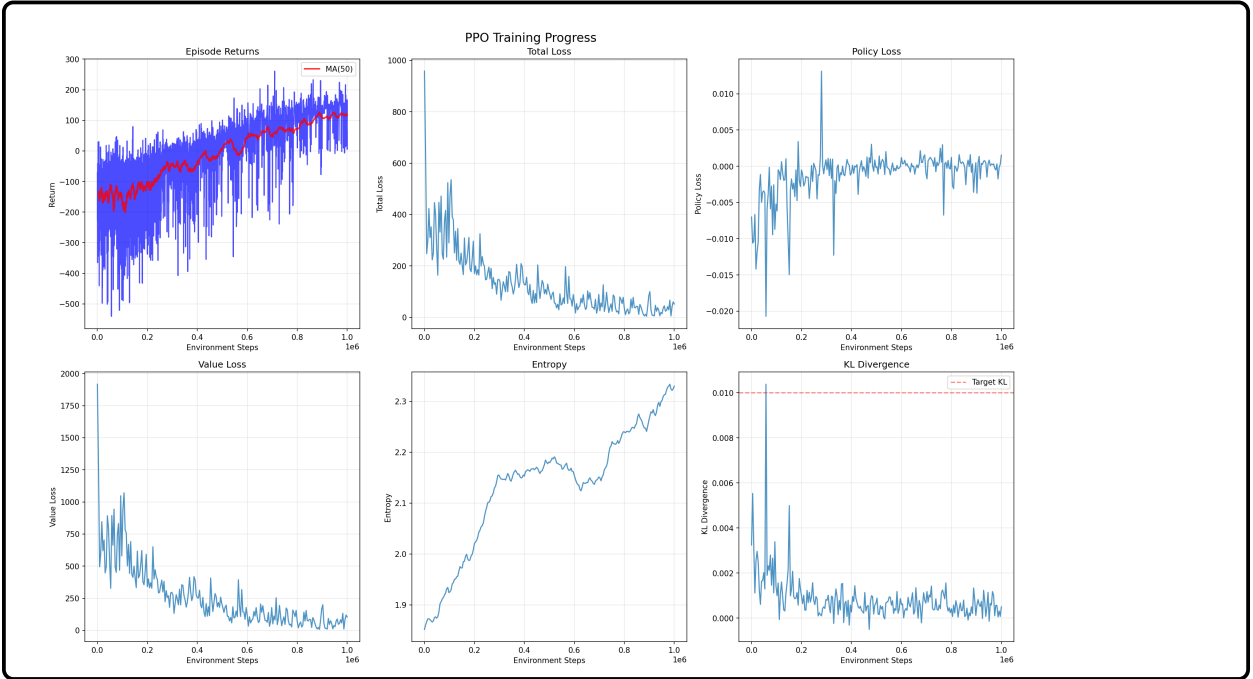
With $\epsilon = 0.3$, learning was faster than 1.4.1 and the ending MA(50) increased from 142.3 to 245.3, and best-checkpoint evaluation increased from 214.9 to 254.2. KL was generally around 0.01, compared with roughly 0.005 for the 1.4.1, so there were larger policy changes. Lastly, last-50 return SD was similar (53.9 versus 51.5).

For positive advantages, clipping removes the incentive to increase r_t beyond $1 + \epsilon$ and for negative advantages, it removes the incentive to decrease r_t below $1 - \epsilon$. Larger ϵ therefore allows stronger changes before clipping comes in, making the realized objective closer to the unclipped policy gradients.

The trade-off is faster learning versus potentially less stable updates in terms of loss spikes and KL divergences. In our case, the run benefited from the larger threshold while remaining relatively stable.

1.5.3 Low Clipping Threshold ($\epsilon = 0.05$)

Plots



Final returns

Final Performance: 137.4 ± 115.9 .

Training: Ending MA(50) = 115.9. Training returns (max/mean/min): all episodes 260.8/ - 73.0/ - 540.8; last 50 224.2/115.9/ - 6.8 (SD 63.4).

Discussion

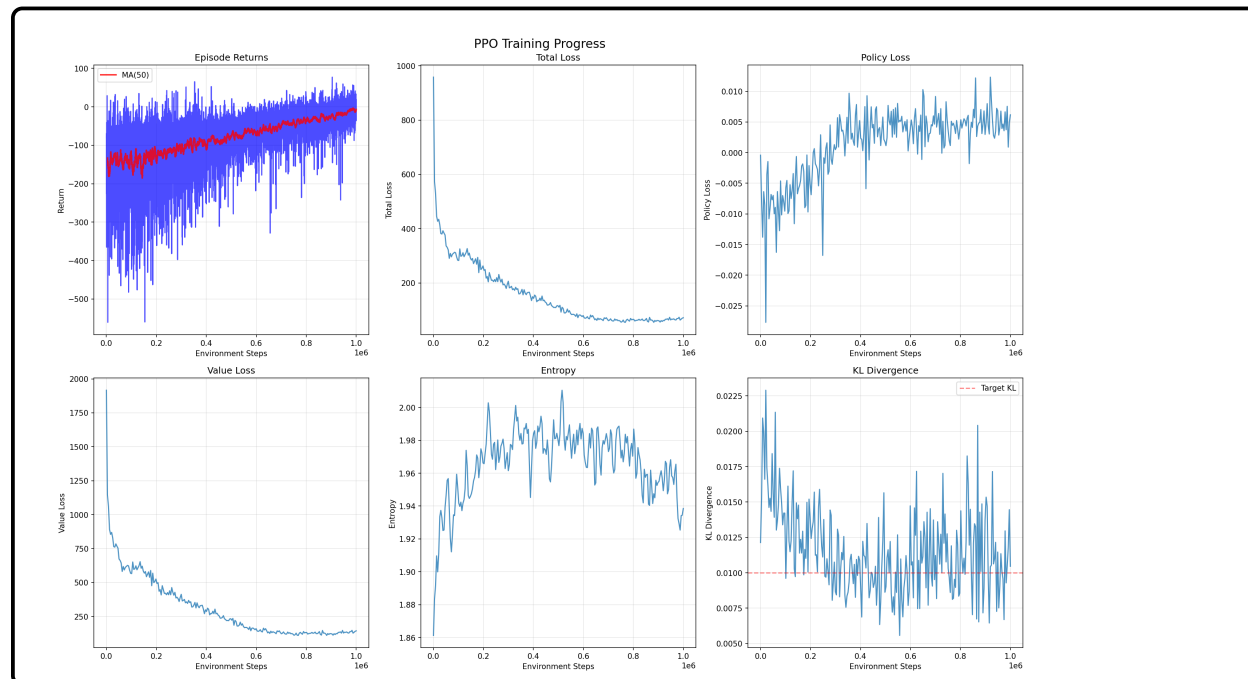
With $\epsilon = 0.05$, learning was slower and the ending MA(50) fell to 115.9 and best-checkpoint evaluation to 137.4, as compared to the 142.3 and 214.9 (resp.) for 1.4.1. After early training, KL was mostly below 0.002, and the policy objective fluctuated less. This indicates more stable policy updates, but does not directly indicate consistent returns since the last-50 return SD increased from 51.5 (1.4.1) to 63.4.

The narrower clipping interval removes incentives for further useful probability changes sooner therefore limiting / slowing down progress. So, smaller updates can improve policy update stability while reducing learning speed and final performance (i.e., with a fixed compute budget).

In higher-dimensional or more complex environments, conservative updates may help avoid disruption owing to higher plausibility for instability. However, excessively tight clipping can still prevent sufficient learning within the available compute budget.

1.6.1 Full Off-Policy PPO

Plots



Final return

Final Performance: -113.3 ± 81.4 .

Training: Ending MA(50) = -6.5 . Training returns (max/mean/min): all episodes $77.3 / -82.5 / -560.8$; last 50 $41.4 / -6.5 / -67.9$ (SD 25.1).

Discussion

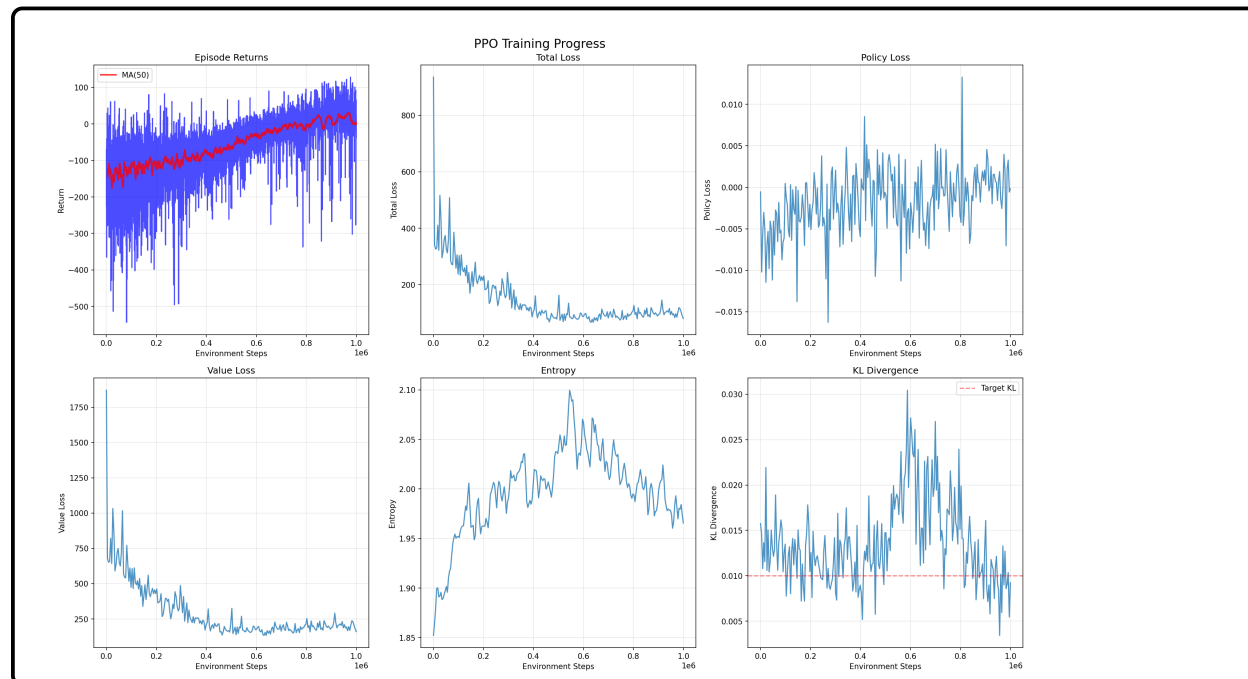
Full replay performed worse than 1.4.1 with the ending MA(50) being -6.5 as compared to 142.3, and the best checkpoint evaluation was -113.3 while 1.4.1 was 214.9. Losses decreased relatively smoothly, and last-50 return SD was only 25.1. However, KL remained around 0.01 with repeated spikes (higher than 0.05 in 1.4.1). Therefore, smooth losses and low return variability do not imply good / efficient learning here with same compute budget.

Older trajectories were collected by policies that chose different actions and visited different states. Their stored advantages and value targets may therefore be less relevant to the current policy. The probability ratio accounts for changes in action probabilities at each sampled state, but does not account for how often the current policy visits that state (**state visitation**). So, replay can direct updates toward states that matter less to the current policy.

Using *more recent trajectories* and recomputing advantages and value targets with some off policy corrections could improve learning.

1.6.2 Half Off-Policy PPO

Plots



Final return

Final Performance: -100.2 ± 55.0 .

Training: Ending MA(50) = 3.3. Training returns (max/mean/min): all episodes 128.1/ - 75.4/ - 543.4; last 50 128.1/3.3/ - 302.1 (SD 92.9).

Discussion

Adding current-policy data modestly improved ending MA(50) from -6.5 to 3.3 , and best-checkpoint evaluation, from -113.3 to -100.2 . However, it did not consistently improve stability since the graph shows larger late KL spikes. Last-50 return SD also increased from 25.1 to 92.9 , with a minimum return of -302.1 .

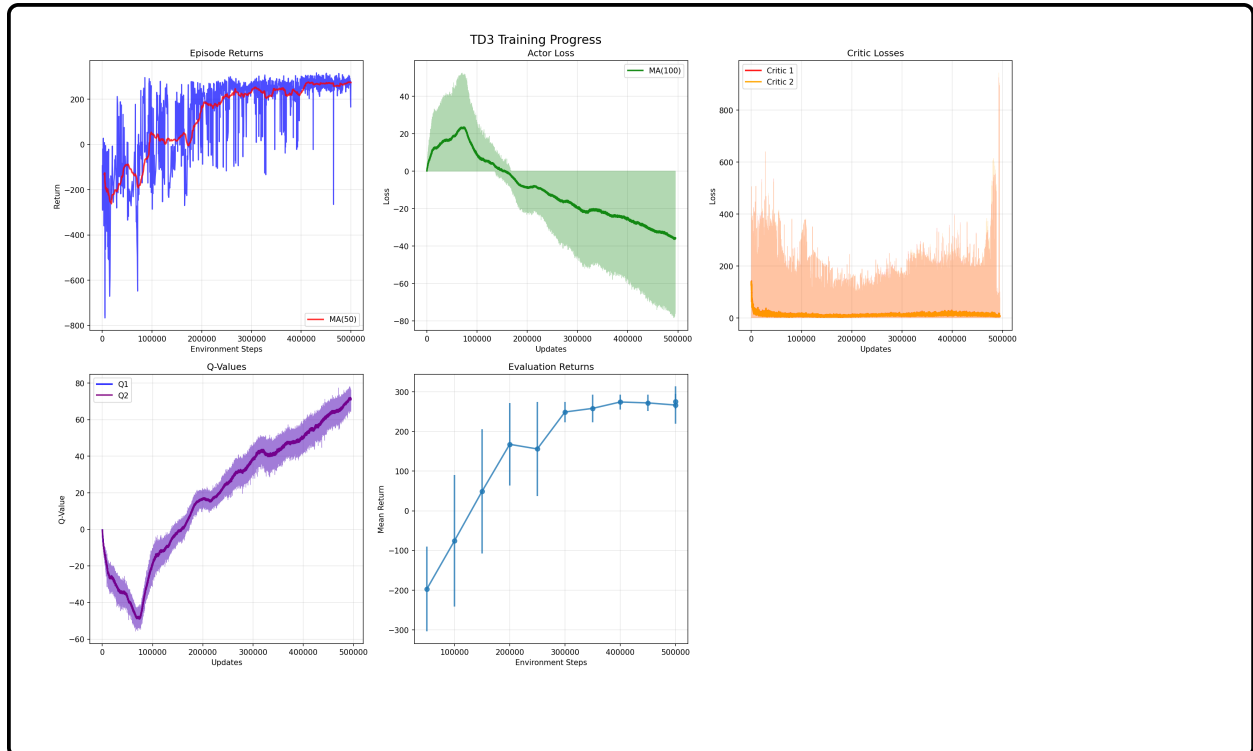
Fresh trajectories provide states and advantage estimates that better reflect the current policy, reducing some historical data mismatch and explaining the improved performance (referred to as state visitation in 1.6.1 answer). Replay still introduces some stale information and both variants remain far below the on-policy baseline (1.4.1). These results show that improved average performance need not come with smaller policy changes or more consistent episode returns.

Problem 2: Twin Delayed DDPG – TD3 (35 pts)

Problems 2.1–2.4 are graded from your code submission (`td3_agent.py`) and have no written component.

2.5 TD3 Evaluation

Plots

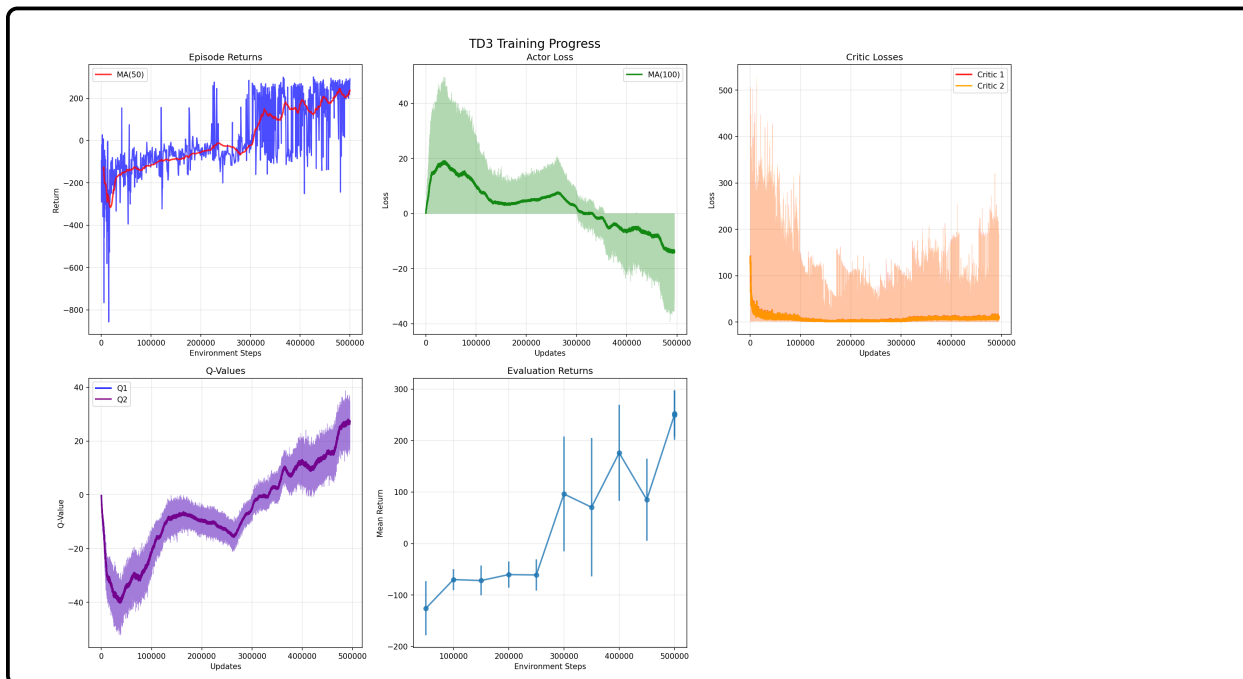


Final performance

Final performance: 275.3 ± 18.8

2.5 Target Policy Smoothing (policy_noise = 0)

Plots



Final performance

Final performance: 253.0 ± 44.6

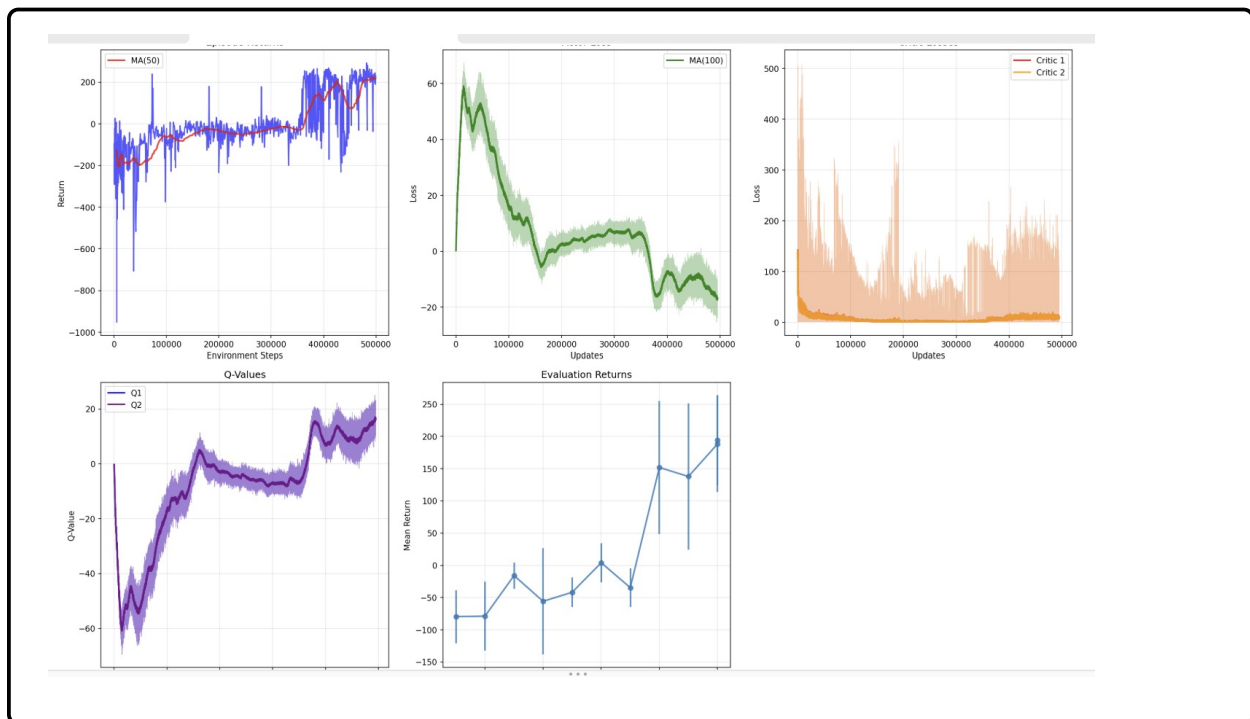
Discussion

From the plots, we observed that removing target policy smoothing made learning slower and less stable. With $\sigma = 0.2$, the evaluation return increased steadily reaching about 250 by 300k steps and then plateauing around 270, with final return of 275.3 ± 18.8 . With $\sigma = 0$, the agent stayed around -60 to -100 for the first 250k steps and then improved more erratically, including a drop from about 175 at 400k steps to 85 at 450k. Its final return was 253.0 ± 44.6 . Importantly, it had more than twice the spread across evaluation episodes.

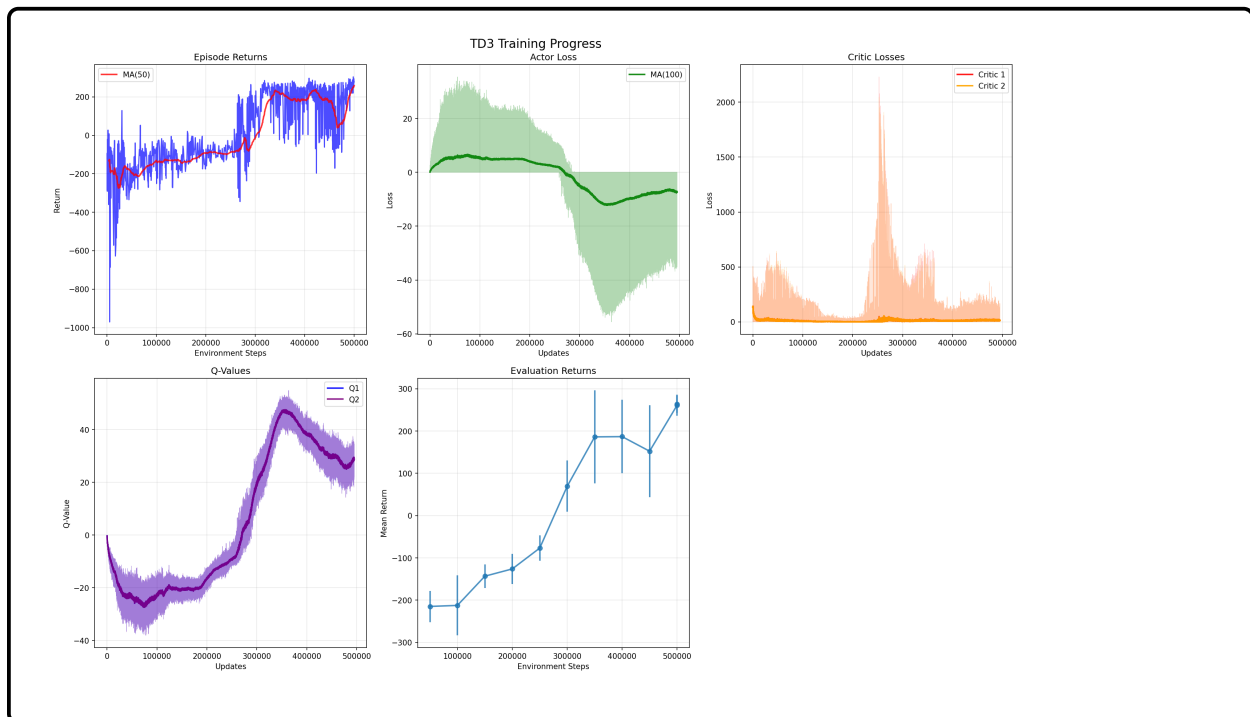
The critic also showed a similar pattern. Q values are smoother with target policy smoothing whereas without it they are less consistent and noisier. This also matches the purpose of TD3's target smoothing. The noise which was added averages out the target Q value over the nearby actions hence, making it harder for the actor to exploit narrow Q function peaks. Overall, the results do suggest that smoothing does improve learning speed and stability rather than clearly improving final performance.

2.6 Policy Delay

Plots: --delay 1



Plots: --delay 4



2.6 (continued) Final performance

policy delay = 1: Final performance: 194.2 ± 69.8
policy delay = 4: Final performance: 263.8 ± 20.9

Discussion

`policy_delay` controls how often the actor and target networks are updated relative to the critics. A larger delay means each actor update uses a more accurate critic, but the actor gets fewer updates and the targets track the online networks more slowly. This is a tradeoff between noisy updates and slow learning.

Final evaluation returns were ≈ 195 (delay 1), ≈ 270 (delay 2, baseline), and ≈ 264 (delay 4). With delay 1, the actor exploits the critic’s errors after every update, so learning was noisy: there was a large early Q dip, a long reward plateau, and non-monotonic evaluation returns.

Delay 4 trained more smoothly. We expected it to learn more slowly, but this did not happen for this seed, since the actor still received about 125k effective updates ($500k/2$). The baseline performed best and we think it because it kept most of the stability benefit while still updating often enough to learn quickly, which matches the TD3 paper’s choice of $d = 2$.

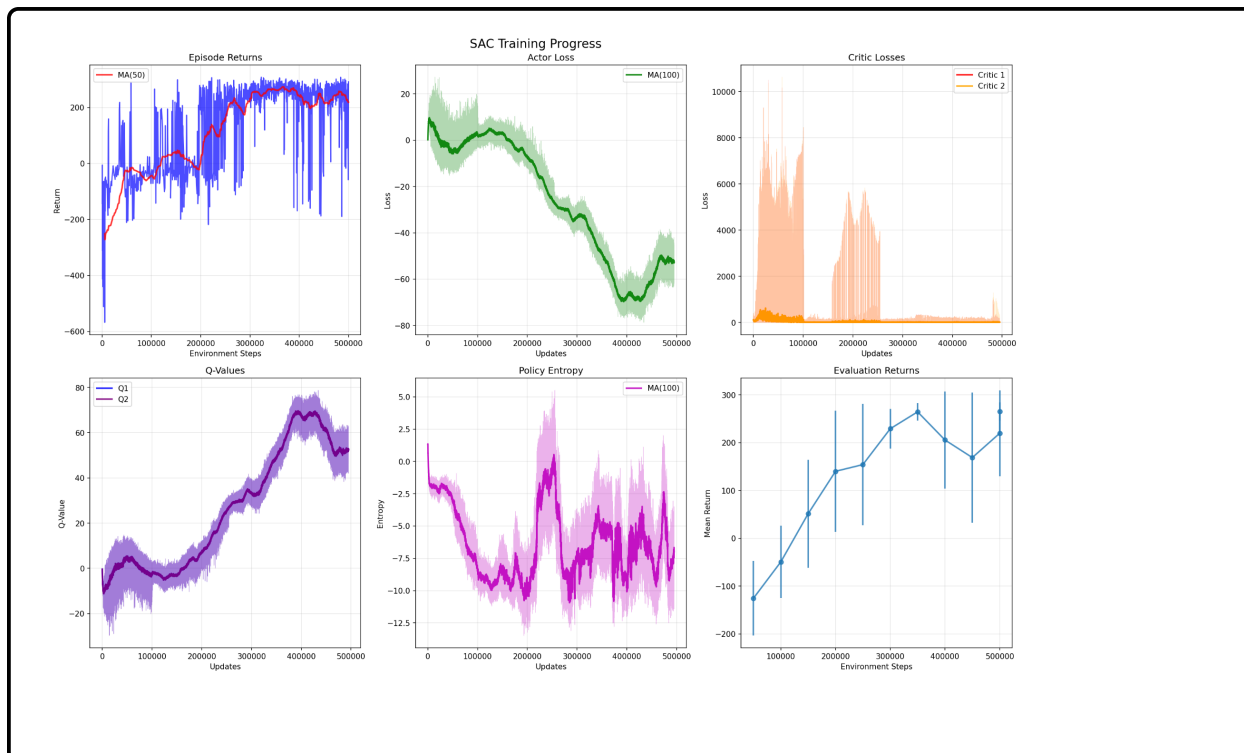
We observed large seed-to-seed variance: in some seeds neither setting learned well, and in others only one did.

Problem 3: Soft Actor-Critic – SAC (15 pts)

Problems 3.1–3.2 are graded from your code submission (`sac_agent.py`) and have no written component.

3.3 SAC Evaluation

Plots



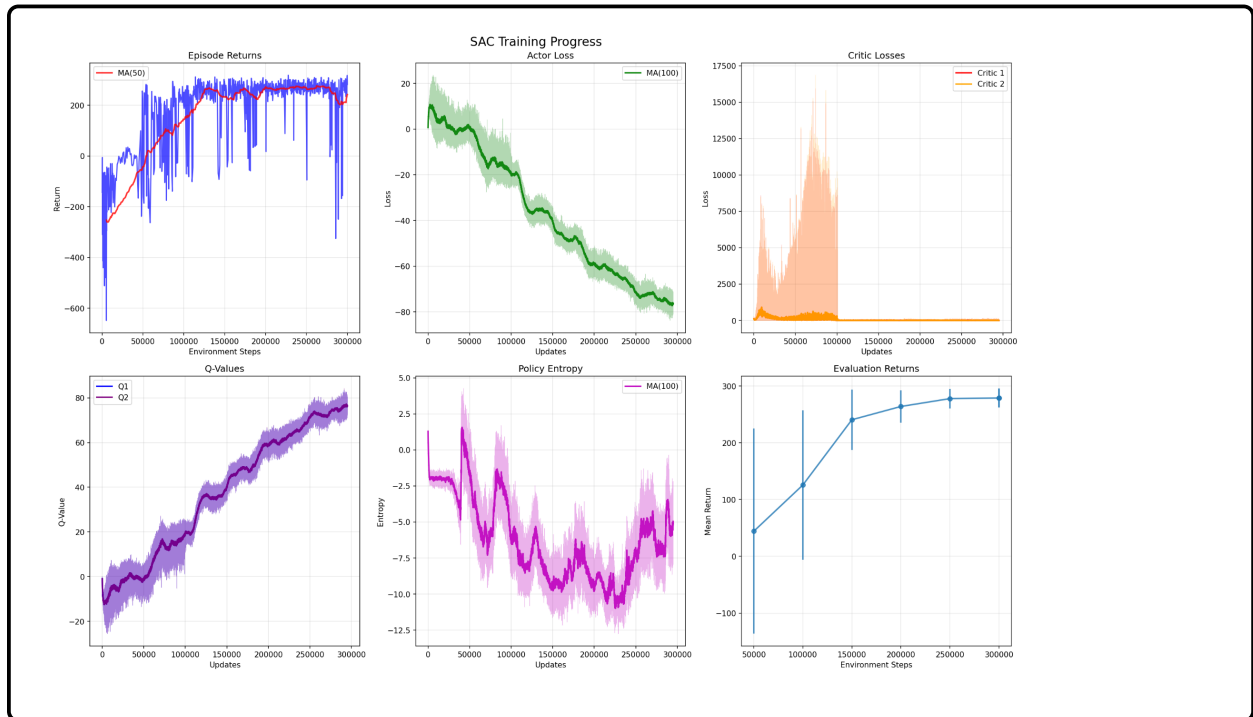
Final performance

Final Performance: 265.3 ± 18.9

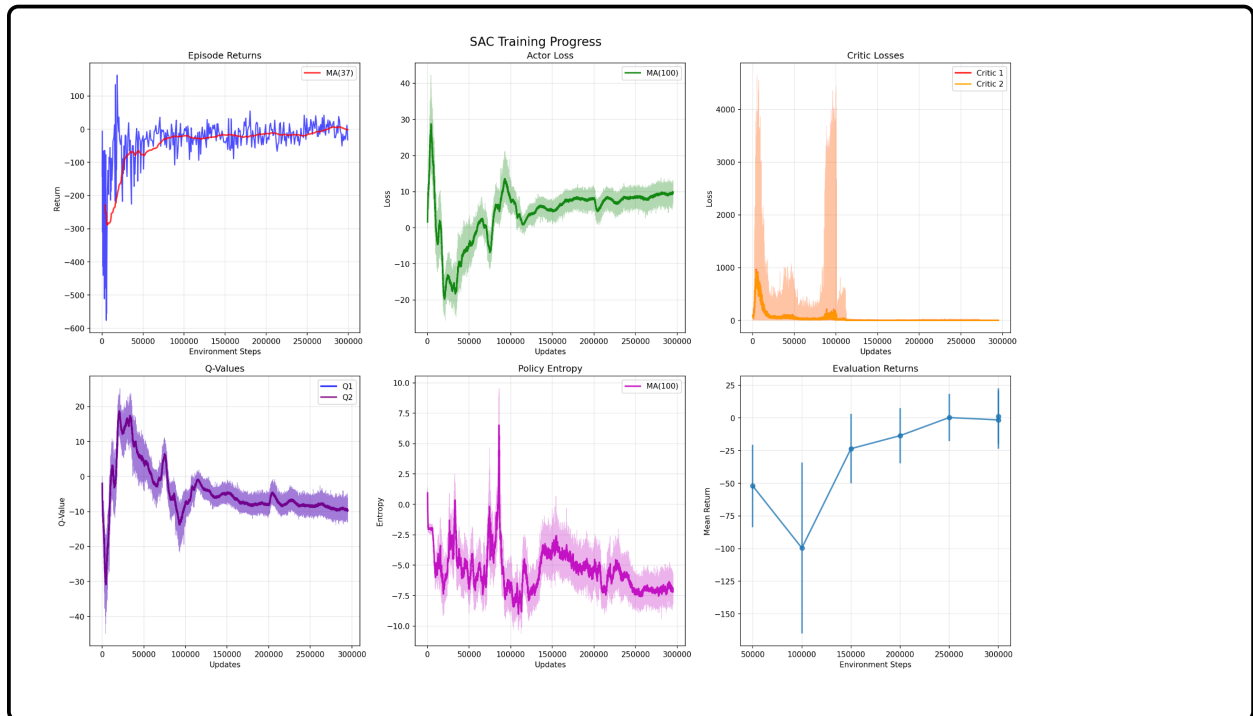
Training: Ending MA(50) = 219.1. Returns (max/mean/min): all episodes 308.2/159.7/ – 567.6; last 50 308.0/219.1/ – 189.5 (SD 103.5) Runtime: 35.3 minutes. The run uses the supplied hyperparameters with optional automatic temperature learning enabled.

3.4 UTD Ratio

Plots: `--utd_ratio 2`



Plots: `--utd_ratio 4`



3.4 (continued) Final return and total run time

UTD 2: Final Performance: 279.1 ± 16.6 . Ending training MA(50) = 242.2 (last-50 SD 113.2). Total runtime: 27.7 minutes.

UTD 4: Final Performance: 1.3 ± 21.4 . Ending training MA(50) = 0.5 (last-50 SD 21.5). Total runtime: 41.7 minutes.

Both runs use 300k environment steps. The UTD 4 plot uses MA(37) but we note MA95) here.

Analysis: sample efficiency vs. stability vs. wall-clock

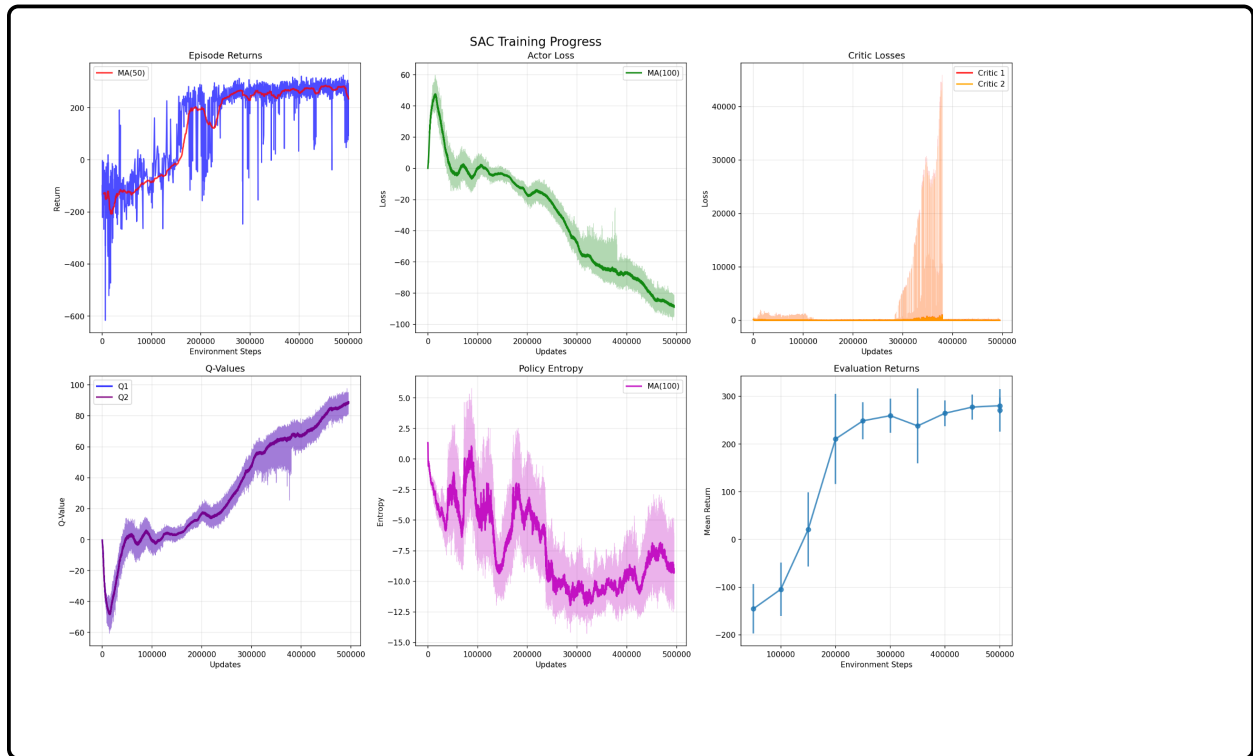
Sample efficiency. UTD N performs N gradient updates per environment step, reusing collected data more often. Using the same budget, MA(50) is 225.6 for the UTD 1, 242.2 for UTD 2, and 0.5 for UTD 4. So, UTD 2 improves sample efficiency in this run, but more updates do not help UTD 4.

Stability. UTD 2 has large early critic-loss spikes, but these decrease as the policy learns high returns, while occasional poor episodes remain (last-50 SD 113.2). UTD 4 also has early loss spikes, then settles near zero return. Its smaller return SD (21.5) and low late critic losses describe consistently poor behavior, rather than successful learning. *We think what happened is that more/repeated replay updates amplified errors in bootstrapped targets.*

Wall-clock. UTD 2 takes 27.7 minutes and UTD 4 takes 41.7 minutes for the same number of steps. The baseline takes 35.3 minutes for 500k steps, so its longer environment interaction budget must be considered. UTD 2 offers the best observed tradeoff, while UTD 4 costs more without improving returns. Results on MacBook Air 2026 with Apple M5 & 24GB memory, and run in parallel.

3.5 Backup Action

Plots



Discussion (one to two sentences)

Using the mean action increased ending training MA(50) from 219.1 to 234.5, reduced last-50 return SD from 103.5 to 81.8, and best performance checkpoint evals from 265.3 ± 18.9 to 270.7 ± 44.8 .

Removing sampling noise during collection can avoid poor exploratory actions, but reduces exploration. The larger evaluation SD and late critic-loss spikes show that this run is not generally more stable, and actor updates still use stochastic samples.

Problem 4: Feedback

Feedback: You can help the course staff improve the course by providing feedback. What was the most confusing part of this homework, and what would have made it less confusing?

N/A

Collaboration: Detail the work division amongst your group below.

All teammates have equally distributed the coding and theory question answering bits.

Time Spent: How many hours did you spend working on this assignment? Your answer will not affect your grade. Please average your answer over all the members of your team.

Alone	5
With teammates	8
With other classmates	0
At office hours	0